



Fig. 4: Circle in the image



Fig. 5: Laplacian edge detection

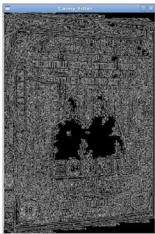


Fig. 6: Edge detection using Canny filter



Fig. 7: Still images



Fig. 8: Face detection output

image which we want to convert and the second denotes the destination variable. The third parameter performs the type of conversion desired. In this case it is from RGB to gray scale.

**Drawing a circle on an image.** You can also edit or manipulate a given image, for example, by drawing a circle in or around it. The `circle.py` code does this.

After running the code you will get output as shown in Fig. 4. From the ninth to the 11th line in `circle.py` code, we have described the parameters of the circle—its thickness, line type and co-ordinates. In line 12, the 'circle' function is used, which has six parameters: the target image on which to draw the circle, the coordinates, the radius of the circle, the colour of the circle, the thickness and the line type.

**The Laplacian of an image.** Often, the Laplace operation is performed on

an image to get the edges. The `laplace.py` code does this.

Fig. 5 shows the output of the code given above. The 'print' function outputs the properties of the image. In the 10th line of `laplace.py` code, the Laplace function is used to obtain the edges. It takes two parameters—the source image and the variable in which to store the processed image. You can use 'saveImage' to save the resultant image, supplying the path, the filename and the variable containing the image. Then load the saved image and display it.

**The Canny filter.** This is a very popular filter, which is used to very accurately detect the edges in an image. The `canny.py` code does this.

Run the `canny.py` code and you will get output as shown in Fig. 6. Convert the input image to gray scale first (see the ninth line), before applying the

Canny filter in the 15th line. This function takes five parameters: the source image, the variable in which to store the processed image, and the rest are the threshold values for this filter.

## Face detection

Now, let us detect faces from still images (refer Fig. 7) using a Haar classifier. The Haar classifier is a pre-made program, which you can train to do operations based on your needs. The more you run the program, the more the Haar classifier learns and trains itself. All Haar classifiers are located in `OpenCV-2.3.0/data/haarcascades`. The `face.py` code does this.

Run the `face.py` code and the output is as shown in Fig. 8. In the ninth line of `face.py` code the Haar classifier is stored which you may use in a variable, as given below:

```
hc=cv.Load("/home/jayneil/haarcascade_frontalface_default.xml")
```

In the 11th line, the 'HaarDetectObjects' function is used to detect the features you want. This function has three parameters.

The first one is the source image, the second the Haar classifier that you want to use and the third one creates a memory storage. Then there is a 'for' loop to draw a rectangle around the portion that you may have identified in the previous stage, using the 'rectangle' function. 'Rectangle' function takes four parameters: the image on which to draw, followed by the two opposite corners of the rectangle, and the last one denotes the colour.

## Serial communication

This section will show you how to use the `pyserial` module, Arduino and, finally, how to build a full-fledged embedded system for image processing. `Pyserial` is a library that provides Python support for serial communication.

Now, let us explore how to use `pyserial` and send data to other devices